

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 1
“Основи функціонального програмування у Python”

Виконав:

студент 2 курсу
студент групи ФЕП-21

Захарчук Адріан

Перевірів:

асис. Остріков Д.А

Ціль:

- зрозуміти, що таке чиста функція і референтна прозорість;
- навчитися виявляти та ізолювати побічні ефекти;
- переписати імперативний фрагмент у функціональному стилі;
- застосувати `typing.Callable` для параметризації обчислень.

Теорія

- **Чиста функція:** детермінована (однаковий вхід $\rightarrow$ однаковий вихід), без змін глобального стану/вхідних даних, без I/O усередині.
- **Референтна прозорість:** вираз можна замінити його значенням без зміни поведінки програми.
- **Побічні ефекти:** I/O (`print`, файл, мережа), мутація вхідних структур, використання/зміна глобалів, залежність від `time`, `random`, `os.environ` тощо.
- **Підхід:** відокремити чисте ядро (перетворення даних) від оболонки ефектів (читання/запис).

Індивідуальний варіант виконання лабораторної роботи

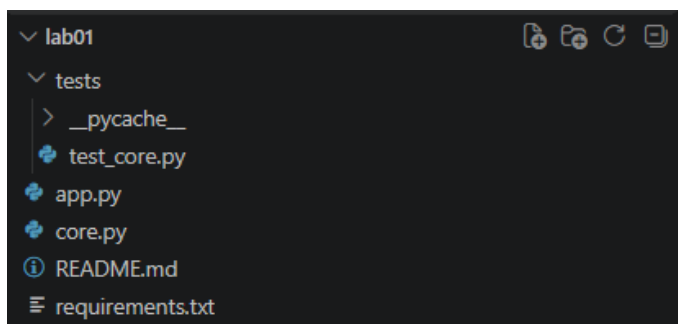
Варіант №5. Бронювання готелю

- Про що завдання: розрахувати вартість проживання гостей у готелі.
- Що маємо на вході: список замовлень, де вказано кількість ночей, ціну за добу, категорію номера та статус оплати/підтвердження.
- Що треба зробити:
 - відібрати тільки підтверджені броні;
 - порахувати базову ціну (ночі помножити на тариф);
 - додати сезонну знижку та туристичний збір;
 - підбити загальну суму виручки.
- Головні правила (функціональний підхід):
 - вхідні дані не змінюємо, для результату створюємо нові словники;
 - ніяких глобальних змінних і `print` усередині обчислень;
 - знижку, податок і перевірку статусу передаємо як окремі змінні функції через `Callable`.
 -

1. Структура проекту та залежності

Проект організовано за модульним принципом відповідно до вимог:

- `core.py` - чисте обчислювальне ядро без побічних дій та глобальних змінних.
- `app.py` - точка входу та оболонка побічних ефектів, що містить консольний друк і конфігурацію бізнес-правил.
- `tests/test_core.py` - набір тестів `pytest` для валідації чистоти, референтної прозорості та незмінності структур даних.
- `requirements.txt` - перелік необхідних бібліотек (`pytest`, `mypy`, `black`, `ruff`).



core.py (Чисте ядро): містить усю бізнес-логіку варіанта №5 без побічних ефектів. Описує модель Booking через TypedDict, чисті функції розрахунку `calculate_base_cost` і створення нових об'єктів `with_total`, а також фабрику `make_booking_processor`, яка конфігурує обробку через `typing.Callable` (фільтр, знижка, туристичний збір).

```

core.py X
core.py
1  """Чисте ядро обробки бронювань готелю (Варіант №5)."""
2
3  from typing import Callable, Dict, Iterable, List, Optional, TypedDict
4
5
6  class Booking(TypedDict, total=False):
7      """Словник опису бронювання готелю."""
8      id: int
9      nights: int
10     price_per_night: float
11     room_type: str
12     status: str # наприклад, 'confirmed', 'cancelled', 'pending'
13     base_cost: float
14     total: float # підсумкова вартість після розрахунку
15
16
17 # Визначення функціональних типів (Callable) для політик
18 SubtotalFn = Callable[[Booking], float]
19 DiscountFn = Callable[[float, Booking], float]
20 CityTaxFn = Callable[[float, Booking], float]
21 FilterFn = Callable[[Booking], bool]
22 BookingProcessorFn = Callable[[Iterable[Booking]], Dict[str, object]]
23
24
25 def calculate_base_cost(booking: Booking) -> float:
26     """Чиста функція: розрахунок базової вартості проживання (ночі * ціна за ніч)."""
27     return round(float(booking["nights"] * booking["price_per_night"]), 2)
28
29
30 def with_total(booking: Booking, base_cost: float, total: float) -> Booking:
31     """Чиста функція: повертає новий словник без мутації вихідного."""
32     return {
33         **booking,
34         "base_cost": base_cost,
35         "total": round(total, 2),
36     }
37
38
39 def default_status_filter(booking: Booking) -> bool:
40     """Чиста функція: фільтрація тільки підтверджених бронювань."""
41     return booking.get("status") == "confirmed"
42

```

```

44  def make_booking_processor(
45      status_filter: FilterFn,
46      apply_discount: DiscountFn,
47      apply_tax: CityTaxFn,
48  ) -> BookingProcessorFn:
49      """Функція вищого порядку (фабрика обробника) для бронювань.
50
51      Повертає чисту функцію-обробник, конфігуровану переданими політиками.
52      """
53      def process(bookings: Iterable[Booking]) -> Dict[str, object]:
54          qualified: List[Booking] = []
55          total_revenue = 0.0
56
57          for b in bookings:
58              # Ізольована перевірка критерію прийнятності
59              if not status_filter(b):
60                  continue
61
62              base_cost = calculate_base_cost(b)
63              # Застосування варіативних політик
64              discounted = apply_discount(base_cost, b)
65              final_total = apply_tax(discounted, b)
66
67              new_booking = with_total(b, base_cost=base_cost, total=final_total)
68              qualified.append(new_booking)
69              total_revenue += final_total
70
71          return {
72              "count": len(qualified),
73              "revenue": round(total_revenue, 2),
74              "bookings": qualified,
75          }
76
77      return process

```

app.py (I/O-оболонка): точка входу застосунку. Задає конкретні правила через лямбда-функції (підтверджений статус, сезонна знижка 15%, диференційований турзбір), збирає тестові бронювання, викликає чистий процесор і виводить звіт у консоль через функцію `render_report`.

```
app.py > ...
1  """Оболонка введення/виведення (I/O) для запуску обробки бронювань."""
2
3  from typing import Dict, List
4  from core import (
5      Booking,
6      CityTaxFn,
7      DiscountFn,
8      FilterFn,
9      make_booking_processor,
10 )
11
12
13 def render_report(result: Dict[str, object]) -> None:
14     """Ізольована функція відображення звіту (I/O)."""
15     print("=" * 60)
16     print("          БІІІ ОБРОБКИ БРОНЮВАНЬ ГОТЕЛЮ")
17     print("=" * 60)
18     print(f"Кількість підтверджених бронювань : {result['count']}")
19     print(f"Загальна виручка готелю          : {result['revenue']:.2f} грн")
20     print("-" * 60)
21     print("Деталізація бронювань:")
22
23     bookings = result.get("bookings", [])
24     if isinstance(bookings, list):
25         for b in bookings:
26             print(
27                 f" ID: {b['id']} | Номер: {b['room_type']:<8} | "
28                 f"Ночей: {b['nights']} | Базова: {b['base_cost']:>8.2f} грн | "
29                 f"Разом: {b['total']:>8.2f} грн"
30             )
31     print("=" * 60)
32
33
34 def main() -> None:
35     # Тестовий набір даних
36     sample_bookings: List[Booking] = [
37         {
38             "id": 101,
39             "nights": 5,
40             "price_per_night": 1200.0,
41             "room_type": "standard",
42             "status": "confirmed",
43         },
44         {
45             "id": 102,
46             "nights": 2,
47             "price_per_night": 2500.0,
48             "room_type": "suite",
49             "status": "cancelled",
50         },
51         {
52             "id": 103,
53             "nights": 8,
54             "price_per_night": 1500.0,
```

```

53         "nights": 8,
54         "price_per_night": 1500.0,
55         "room_type": "deluxe",
56         "status": "confirmed",
57     },
58     {
59         "id": 104,
60         "nights": 1,
61         "price_per_night": 1000.0,
62         "room_type": "standard",
63         "status": "pending",
64     },
65     {
66         "id": 105,
67         "nights": 12,
68         "price_per_night": 3000.0,
69         "room_type": "suite",
70         "status": "confirmed",
71     },
72 ]
73
74 # Функціональні політики Варіанта №5:
75 # 1. Фільтрація: тільки підтверджені
76 is_confirmed: FilterFn = lambda b: b.get("status") == "confirmed"
77
78 # 2. Сезонна/тривала знижка: 15% при тривалості понад 7 ночей, інакше 0%
79 seasonal_discount: DiscountFn = lambda cost, b: (
80     cost * 0.85 if b.get("nights", 0) >= 7 else cost
81 )
82
83 # 3. Туристичний збір: 5% для люксів/делюксів, 2% для стандарту
84 tourist_tax: CityTaxFn = lambda cost, b: cost * (
85     1.05 if b.get("room_type") in ("suite", "deluxe") else 1.02
86 )
87
88 # Збирання конфігурованого процесора через функцію вищого порядку
89 processor = make_booking_processor(
90     status_filter=is_confirmed,
91     apply_discount=seasonal_discount,
92     apply_tax=tourist_tax,
93 )
94
95 # Виконання чистих обчислень
96 result = processor(sample_bookings)
97
98 # Відображення звіту (I/O)
99 render_report(result)
100
101
102 if __name__ == "__main__":
103     main()
104

```

tests/test_core.py (Тести): модуль автотестів на базі pytest. Перевіряє відсутність мутацій вхідних списків і словників (деерсору), детермінованість (референтну прозорість) та точність математичних розрахунків при заміні Callable-політик.

tests > test_core.py > ...

```
1  """Тести для перевірки чистого функціонального ядра (pytest)."""
2
3  from copy import deepcopy
4  import pytest
5  from core import (
6      Booking,
7      calculate_base_cost,
8      default_status_filter,
9      make_booking_processor,
10     with_total,
11 )
12
13
14 @pytest.fixture
15 def sample_bookings() -> list[Booking]:
16     return [
17         {
18             "id": 1,
19             "nights": 3,
20             "price_per_night": 1000.0,
21             "room_type": "standard",
22             "status": "confirmed",
23         },
24         {
25             "id": 2,
26             "nights": 10,
27             "price_per_night": 2000.0,
28             "room_type": "suite",
29             "status": "confirmed",
30         },
31         {
32             "id": 3,
33             "nights": 4,
34             "price_per_night": 1500.0,
35             "room_type": "deluxe",
36             "status": "cancelled",
37         },
38     ]
39
40
41 def test_calculate_base_cost():
42     booking: Booking = {
43         "id": 1,
44         "nights": 4,
45         "price_per_night": 1250.0,
46         "room_type": "standard",
47         "status": "confirmed",
48     }
49     assert calculate_base_cost(booking) == 5000.0
50
```

```

48     },
49     assert calculate_base_cost(booking) == 5000.0
50
51
52 def test_with_total_no_mutation():
53     original: Booking = {
54         "id": 1,
55         "nights": 2,
56         "price_per_night": 500.0,
57         "room_type": "standard",
58         "status": "confirmed",
59     }
60     original_copy = deepcopy(original)
61
62     updated = with_total(original, base_cost=1000.0, total=1020.0)
63
64     # Вихідний об'єкт не змінився
65     assert original == original_copy
66     assert "total" not in original
67     # Отримано новий об'єкт із правильними даними
68     assert updated["total"] == 1020.0
69     assert updated["base_cost"] == 1000.0
70     assert updated["id"] == original["id"]
71
72
73 def test_referential_transparency(sample_bookings):
74     """Перевірка референтної прозорості: однаковий вхід дає однаковий вихід."""
75     processor = make_booking_processor(
76         status_filter=default_status_filter,
77         apply_discount=lambda cost, b: cost * 0.9,
78         apply_tax=lambda cost, b: cost * 1.05,
79     )
80
81     result_1 = processor(sample_bookings)
82     result_2 = processor(sample_bookings)
83
84     assert result_1 == result_2
85
86
87 def test_no_input_mutation(sample_bookings):
88     """Перевірка повної відсутності мутацій вихідного списку."""
89     original = deepcopy(sample_bookings)
90     processor = make_booking_processor(
91         status_filter=default_status_filter,
92         apply_discount=lambda cost, b: cost,
93         apply_tax=lambda cost, b: cost,
94     )
95
96     _ = processor(sample_bookings)
97     assert sample_bookings == original
98

```

```

96     _ = processor(sample_bookings)
97     assert sample_bookings == original
98
99
100 def test_callable_policies_and_filtering(sample_bookings):
101     """Перевірка правильного застосування динамічних політик."""
102     # Політика: знижка 200 грн на бронювання тривалістю понад 5 ночей, збір 50 грн фіксовано
103     discount_policy = lambda cost, b: cost - 200.0 if b["nights"] > 5 else cost
104     tax_policy = lambda cost, b: cost + 50.0
105
106     processor = make_booking_processor(
107         status_filter=lambda b: b["status"] == "confirmed",
108         apply_discount=discount_policy,
109         apply_tax=tax_policy,
110     )
111
112     res = processor(sample_bookings)
113
114     # Тільки 2 підтверджених бронювання мають потрапити до вибірки
115     assert res["count"] == 2
116
117     # Перевірка розрахунків:
118     # Booking 1: 3 ночі * 1000 = 3000 -> знижка 0 -> +50 збір = 3050.0
119     # Booking 2: 10 ночей * 2000 = 20000 -> знижка 200 = 19800 -> +50 збір = 19850.0
120     # Разом: 3050 + 19850 = 22900.0
121     assert res["revenue"] == 22900.0
122

```

requirements.txt (Залежності): перелік інструментів розробки - тестовий фреймворк pytest, аналізатор типів mypy та літери коду.

```
requirements.txt X
requirements.txt
1  pytest>=7.4.0
2  mypy>=1.5.0
3  black>=23.7.0
4  ruff>=0.0.285
```

README.md (Документація): опис реалізованого варіанта №5, інструкція зі створення оточення, встановлення залежностей і запуску скрипта та тестів.

```
README.md X
README.md > abc # Лабораторна робота №1. Основи функціонального програмування у Python
1  # Лабораторна робота №1. Основи функціонального програмування у Python
2
3  ## Варіант №5: Бронювання готелю
4  * **Вхідні дані:** бронювання готелю з полями `id`, `nights` (кількість ночей), `price_per_night` (ціна за ніч), `room_type` (тип номера) та `status` (статус замовлення).
5  * **Основне завдання:** обчислити вартість проживання, сезонну/тривалу знижку та туристичний збір.
6  * **Функціональні вимоги:**
7      1. Реалізовано чисте обчислювальне ядро без побічних ефектів (`print`, глобальний стан, системні виклики).
8      2. Всі вхідні структури є незмінними (створюються нові копії словників `with_total`).
9      3. Політики фільтрації за статусом, нарахування знижок та туристичного збору реалізовані через `typing.Callable`.
10     4. Використано функцію вищого порядку `make_booking_processor` для створення сконфігурованого конвеєра обробки.
11     5. Введення/виведення (I/O) винесено окремо у файл `app.py`.
12
13  ## Встановлення та запуск
14
15  1. Створити та активувати віртуальне середовище:
16  ```bash
17  python -m venv .venv
18  # Для Linux/macOS:
19  source .venv/bin/activate
20  # Для Windows:
21  .venv\Scripts\activate
```

2. Результати виконання програми

При запуску програми `python app.py` через термінал вхідні дані успішно проходять фільтрацію. Непідтверджені замовлення (`cancelled`, `pending`) відсіюються, а для підтверджених коректно обраховуються знижки, збори та формується підсумковий баланс готелю.

```
=====
                        ЗВІТ ОБРОБКИ БРОНЮВАНЬ ГОТЕЛЮ
=====
Кількість підтверджених бронювань : 3
Загальна виручка готелю           : 48960.00 грн
=====
Деталізація бронювань:
ID: 101 | Номер: standard | Ночей: 5 | Базова: 6000.00 грн | Разом: 6120.00 грн
ID: 103 | Номер: deluxe   | Ночей: 8 | Базова: 12000.00 грн | Разом: 10710.00 грн
ID: 105 | Номер: suite    | Ночей: 12 | Базова: 36000.00 грн | Разом: 32130.00 грн
=====
PS D:\lab01>
```

3. Автоматичне тестування (tests/test_core.py)

Для підтвердження математичної та функціональної коректності створено автоматичні тести під керуванням pytest

```
PS D:\lab01> python -m pytest -v
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\user\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz
5n2kfra8p0\python.exe
cachedir: .pytest_cache
rootdir: D:\lab01
collected 5 items

tests/test_core.py::test_calculate_base_cost PASSED [ 20%]
tests/test_core.py::test_with_total_no_mutation PASSED [ 40%]
tests/test_core.py::test_referential_transparency PASSED [ 60%]
tests/test_core.py::test_no_input_mutation PASSED [ 80%]
tests/test_core.py::test_callable_policies_and_filtering PASSED [100%]

===== 5 passed in 0.03s =====
PS D:\lab01>
```

4. Статичний аналіз типів (муру)

Код пройшов валідацію статичного типізатора муру

```
PS D:\lab01> python -m mypy core.py app.py
Success: no issues found in 2 source files
PS D:\lab01>
```

5. Висновок: під час виконання лабораторної роботи були виконані вимоги основ функціонального програмування у Python і переписано типовий імперативний код у функціональному стилі. Для свого варіанта («Бронювання готелю») я повністю розділив програму на дві частини: у core.py виніс чисте ядро розрахунків, де позбувся глобальних змінних, мутацій вхідних даних та викликів print, а самі правила знижок, зборів і фільтрації зробив гнучкими завдяки typing.Callable та функції-фабриці. Увесь консольний вивід та запуск зосередив окремо в app.py. Написані pytest-тести підтвердили референтну прозорість функцій і цілісність початкових списків, а перевірка муру засвідчила коректність усіх типів.